Data Science for Software Engineering

Assignment 1 Report

Hadoop Yarn

Professor: Prof. Vaibhav Chaudhari

Group 10

Sujit Jagdishbhai Maniya (4038595)

Mihir Sane (4086665)

Aryan Solanki (4106036)

Jenish Jai (4090405)

Indudhara S (4106839)

1 Introduction

Apache Hadoop YARN (Yet Another Resource Negotiator) serves as the core cluster management layer for the Hadoop ecosystem [1]. It is responsible for managing computing resources and scheduling applications across distributed nodes. This study focuses specifically on the **ResourceManager Scheduler Capacity** component, located within the org.apache.hadoop.yarn.server.resourcemanager .scheduler.capacity package. As the ResourceManager is the central authority for resource arbitration, the Capacity Scheduler plays a vital role in enabling multi-tenancy, ensuring that system resources are shared efficiently among multiple organizations while maintaining guaranteed capacities.

The primary motivation for this research stems from the challenge of **architectural drift**. Large-scale software systems like Hadoop YARN undergo continuous evolution, which often leads to a divergence between the originally intended design and the actual implementation. This drift complicates system maintenance, increases technical debt, and obscures the logical boundaries between components [2]. By performing architectural recovery, we aim to extract high-level structures from the source code to evaluate the current modularity of the system. This project specifically investigates how automated clustering algorithms can reconstruct these boundaries and how emerging Large Language Model (LLM) technologies can assist in interpreting and documenting the resulting architecture.

1.1 Background

1.1.1 Software Clustering and Structural Analysis

Architectural recovery typically utilizes software clustering to partition a system's classes into cohesive subsystems. This study leverages the **ARCADE** tool suite to implement and compare three foundational algorithms [3]:

- **ACDC (Algorithm for Comprehension-Driven Clustering):** A structural clustering algorithm that groups classes based on common subsystem patterns, such as "support" or "central" components, to mimic human architectural reasoning [4].
- **WCA (Weighted Conglomerative Algorithm):** A hierarchical clustering approach that uses similarity measures like the Unweighted External Measure (UEM) to group entities based on shared dependencies.
- **LIMBO (LIkelihood Based Optimization):** An information-theoretic algorithm that treats clustering as a data compression problem, aiming to minimize the loss of mutual information (IL) between software entities [5].

Evaluating these algorithms requires objective metrics. The **a2a (Architecture-to-Architecture)** metric is used to determine structural similarity between outputs, while the **Cvg (Coverage)** metric measures the degree of overlap between cluster boundaries. Literature indicates that different algorithms often capture distinct "views" of an architecture [6], necessitating a comparative approach.

1.1.2 LLMs in Architectural Recovery

Recent research has begun exploring Large Language Models (LLMs) to provide the semantic context that structural clustering lacks [7]. While clustering identifies groups, LLMs like **ByteDance-Seed/Seed-Coder-8BInstruct** and **zai-org/GLM-5** can analyze the underlying source code to generate human-readable titles and summaries [8]. This integration represents a shift toward "semantic architectural recovery," where AI-driven qualitative analysis complements traditional quantitative structural data to provide a holistic view of software systems.

2 Study Design

The study aims to address the following questions regarding the intersection of structural analysis and artificial intelligence:

2.1 RQ1: How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?

The research was executed through a systematic workflow spanning several weeks, focusing on the Hadoop YARN ResourceManager Capacity Scheduler.

2.1.1 Phase 1: Dependency Extraction and Filtering

The initial step involved compiling the Hadoop YARN ResourceManager module to generate the compiled `.jar` file. Using ARCADE JavaParser with `MASTER_PREFIX=org.apache.hadoop`, we generated a Hadoop-only master RSF file with 11,972 dependency lines. We then filtered this file for the `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity` package, producing a focused RSF file with 1,812 dependency lines. This newer pipeline avoids clustering external Java, Jackson, SLF4J, and other library classes.

2.1.2 Phase 2: Multi-Algorithm Clustering

To compare different architectural "views," we applied three distinct clustering algorithms to our focused dataset:

1. **ACDC:** A pattern-driven structural clustering tool.
2. **WCA:** A hierarchical algorithm utilizing the Unweighted External Measure (UEM) similarity measure.
3. **LIMBO:** An information-theoretic algorithm utilizing the Information Loss (IL) similarity measure.

Each clustering output now contains 466 containment lines, representing the Hadoop classes retained after the cleaner package-filtered dependency extraction.ACDC produced 7 clusters, while WCA and LIMBO each produced 50 clusters.

2.1.3 Phase 3: Comparative Evaluation

We conducted a quantitative evaluation using ARCADE's `a2a` and `Cvg` metric tools [6]. The `a2a` (Architecture-to-Architecture) scores indicated that WCA and ACDC produced the most similar structural results (88.391), while the `Cvg` (Coverage) scores indicated that WCA and ACDC produced the most similar structural results (0.1429).The specific boundaries defined by each algorithm vary significantly, highlighting the need for qualitative analysis.

2.1.4 Phase 4: LLM Prototyping and Scaling

The final phase incorporates LLMs to interpret the recovered clusters and generate semantic architectural descriptions [9]. We successfully prototyped our prompting logic using the **ByteDance-Seed/Seed-Coder-8BInstruct-Instruct** model on a T4 GPU in Google Colab. Future work involves scaling these prompts to the **zai-org/GLM-5** heavyweight model on an HPC cluster to generate comprehensive architectural summaries for the full Hadoop Yarn component.

2.1.5 Phase 5: ARC Semantic-Structural Clustering Implementation

We implemented ARC clustering to combine semantic source-code embeddings with structural dependency data. We utilized the **jinaai/jina-code-embeddings-0.5b** model with 4-bit quantization to generate semantic vectors from the Hadoop YARN Capacity Scheduler Java files, calculating similarity via cosine distance. Structural matrices were simultaneously built from the Week 1 RSF class dependencies. These matrices were fused using `ALPHA` $= 0.5$ to balance both architectural facets equally. Finally, we applied Agglomerative Clustering to group the components into 10 target clusters, exporting the results to **group10_week3_arc_clusters.rsf**.

The ARC output produced 99 containment lines distributed across 10 clusters. This output was then compared against the Week 1 ACDC, WCA, and LIMBO clustering outputs using the **run_week3_arc.sh** script. The comparison used the same ARCADE A2A and Cvg metrics, allowing us to evaluate how the ARC semantic-structural clustering result differs from the traditional structural clustering outputs.

2.2 RQ2: How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

The hierarchical summarization pipeline successfully completed all 101 leaf-level summaries and produced the final **final_architectural_summary.txt** as the root-level architectural overview.

At the leaf level, flat prompts generated technically correct but shallow descriptions, mostly summarizing method signatures and variable names without connecting the class to any broader purpose. Contextual hierarchical prompts produced noticeably richer output for example, summaries for classes inside `allocator` and `preemption` subdirectories clearly described their role within the Capacity Scheduler rather than just listing what methods do.

At the branch level, this difference became more visible. Branch summaries built from contextual prompts described relationships between classes within a subsystem, while those from flat prompts largely repeated class-level details without drawing higher-level connections.

The final root summary reflected this gap clearly the contextual approach produced a coherent overview of how the Capacity Scheduler manages queue hierarchies, resource limits, and tenant-level scheduling, while the flat approach produced a loosely connected list of component descriptions.

3 Results

The results obtained from the applied methods provide an initial understanding of the structural patterns within the Hadoop YARN Capacity Scheduler component, focusing on how different algorithms group the classes and the structural similarity of their recovered architectures.

3.1 RQ1: How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?

Evaluation via ARCADE metrics reveals that ACDC and WCA produced the most structurally similar architectures with an A2A score of 88.391, followed by WCA vs. Limbo (85.794) and Limbo vs. ACDC (83.342). However, coverage (Cvg) values are predominantly 0.0, indicating almost no direct class-level overlap between the recovered clusters across different algorithms. The sole exception is an asymmetric, minimal overlap between ACDC and WCA (0.1429 / 0.02). This highlights that while ACDC and WCA align closely on overall architectural shape, all three algorithms still establish significantly different component boundaries.

Table 1: Pairwise Comparison of Clustering Algorithms

Pair	**A2A**	**Cvg**
WCA vs ACDC	88.391	0.1429 / 0.02
WCA vs Limbo	85.794	0.0 / 0.0
Limbo vs ACDC	83.342	0.0 / 0.0

The ARC clustering output was compared against the Week 1 structural clustering outputs. ARC produced 99 containment lines across 10 clusters. The A2A scores between ARC and the traditional outputs were substantially lower: ARC vs. ACDC scored 26.520, ARC vs. Limbo scored 26.227, and ARC vs. WCA scored 26.061. These lower scores indicate that ARC produces a significantly different architectural view, driven by the integration of semantic source-code embeddings with structural dependency data.

The Coverage results for ARC were similarly low. ARC vs. ACDC and ARC vs. WCA yielded 0.0 Coverage in both directions, while ARC vs. Limbo showed a minor asymmetric overlap of 0.1 from ARC to Limbo and 0.02 from Limbo to ARC. These metrics reinforce that ARC captures an entirely different architectural perspective of the Capacity Scheduler component compared to traditional structural algorithms.

Table 2: Pairwise Comparison of ARC against Structural Clustering Algorithms

Pair	**A2A**	**Cvg**
ARC vs ACDC	26.520	0.0 / 0.0
ARC vs Limbo	26.227	0.1 / 0.02
ARC vs WCA	26.061	0.0 / 0.0

3.2 RQ2: How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

The hierarchical summarization pipeline successfully processed all **101 .java files** distributed across the subdirectories of the `capacity` module. Each file was summarized individually in Stage 1, producing 101 leaf-level summaries saved in the mirrored `summaries/` directory. All 101 jobs completed successfully with no failures.

At the leaf level, flat prompts generated technically correct but shallow descriptions, mostly summarizing method signatures and variable names without connecting the class to any broader purpose. Contextual hierarchical prompts produced noticeably richer output summaries for classes inside `allocator` and `preemption` subdirectories clearly described their role within the Capacity Scheduler rather than just listing what individual methods do.

At the branch level, this difference became more visible. Branch summaries built from contextual prompts described relationships between classes within a subsystem, while those from flat prompts largely repeated class-level details without drawing any higher-level connections.

The final **final_architectural_summary.txt** reflected this gap clearly the contextual approach produced a coherent overview of how the Capacity Scheduler manages queue hierarchies, resource limits, and tenant-level scheduling, while the flat approach produced a loosely connected list of component descriptions without meaningful architectural insight.

3.2.1 Hypothetical Results of Prompting Techniques

Simple Zero-Shot Prompting: Generated a basic summary identifying the role of `CSMaxRunningAppsEnforcer` in enforcing application limits. However, the output lacked architectural structure and omitted important details such as dependencies and component interactions.

One-Shot Prompting: Produced a more structured summary describing the component's functionality, scheduling logic, and key dependencies (`CapacityScheduler`, `CSQueue`, `FiCaSchedulerApp`). The output was more consistent and suitable for architectural documentation.

Chain-of-Thought Prompting: Generated detailed reasoning before summarization, accurately capturing queue/user limit enforcement and scheduler interactions, but with higher computational overhead.

Instruction-Based Zero-Shot Prompting (Chosen Method): Hypothetically provided the best balance of quality and efficiency, generating concise, structured summaries that captured component responsibilities, dependencies, and subsystem relationships for Hadoop YARN architectural recovery.

4 Discussion

To address the first research question, we interpret the observed clustering outcomes in relation to architectural accuracy and component identification. The following discussion examines how effectively the applied algorithms capture meaningful structural boundaries within the Hadoop Yarn system.

4.1 RQ1: How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?

4.1.1 Architectural Isolation of the Capacity Scheduler

One of the more important results from the extraction phase was how much the dependency graph was reduced after applying the Hadoop project-level package filter. Using `MASTER_PREFIX=org.apache.hadoop`, the master RSF contained **11,972 dependency lines**. After filtering for the `org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity` package, the focused RSF contained **1,812 dependency lines**.

This reduction shows that the Capacity Scheduler is a focused part of the larger ResourceManager module. The updated extraction is also cleaner because it keeps Hadoop project classes while excluding external Java, Jackson, SLF4J, and other library/API dependencies from the clustering input. This is important because the assignment is about recovering the architecture of Hadoop Yarn, not clustering third-party dependency classes.

Although the focused RSF is smaller, the remaining 1,812 dependency lines still show that the Capacity Scheduler is internally complex. Its classes interact across queue hierarchies, resource limits, scheduling policies, and tenant-level capacity management. This suggests that the component has a clear external boundary, but a rich and tightly connected internal structure.

4.1.2 What the Identical Containment Counts Actually Mean

After running ACDC, WCA, and Limbo on the focused RSF, each algorithm produced **466 containment lines**. At first, this may look like the algorithms produced the same result, but the identical line count only means that they clustered the same set of input classes.

The actual cluster structures are different. ACDC produced **7 clusters**, while WCA and Limbo each produced **50 clusters**. This means the algorithms made different architectural decisions even though they worked on the same dataset. The containment count tells us how many class-to-cluster assignments exist, but it does not tell us whether the same classes were grouped together.

Therefore, the real comparison must come from the A2A and Cvg metrics, which show how similar or different the recovered architectures are.

4.1.3 Reading the A2A and Cvg Results Together

The updated A2A results show that **ACDC and WCA** are the most structurally similar pair, with an A2A score of **88.391**. Limbo and WCA follow with a score of **85.794**, while ACDC and Limbo have a score of **83.342**. These scores suggest that all three algorithms recover architectures with some structural similarity, but

ACDC and WCA are the closest pair in the current output.

The Cvg results provide a more detailed view of class-level overlap. Most Coverage values are still **0.0**, showing that the algorithms often disagree on exact class-to-cluster composition. However, ACDC and WCA are not completely disconnected: ACDC to WCA has a Coverage score of **0.1429**, and WCA to ACDC has a Coverage score of **0.02**. This means there is a small amount of overlap between ACDC and WCA, but it is limited and asymmetric.

This difference between A2A and Cvg is important. A2A measures the overall architectural shape, while Cvg measures whether the same classes are grouped together. Two clusterings can appear structurally similar while still disagreeing on the exact boundaries of individual clusters.

4.1.4 ARC Clustering and Semantic-Structural Comparison

In Week 3, we also applied ARC clustering by combining semantic similarity from code embeddings with structural similarity from the RSF dependency data. Unlike ACDC, WCA, and Limbo, which mainly rely on structural dependency information, ARC attempts to include semantic meaning from the source code itself.

The ARC output produced **99 containment lines** and **10 clusters**. When compared with the Week 1 clustering outputs, the A2A scores were much lower: ARC vs ACDC scored **26.520**, ARC vs Limbo scored **26.227**, and ARC vs WCA scored **26.061**. These low scores show that ARC produced a substantially different architectural view from the traditional ARCADE clustering algorithms.

The Coverage results were also mostly low. ARC vs ACDC and ARC vs WCA both produced **0.0** Coverage in both directions. ARC vs Limbo showed a small amount of overlap, with **0.1** from ARC to Limbo and **0.02** from Limbo to ARC. This suggests that ARC captures a different perspective on the Capacity Scheduler architecture, likely because it combines semantic information from the code with structural dependency information.

4.1.5 LLM Initial Findings

The early Colab test with **ByteDance-Seed/Seed-Coder-8B-Instruct** was useful for validating the role of LLMs in architectural understanding. Presented with focused Java code, the model was able to explain class responsibilities, describe important variables, and summarize method logic in plain language.

This indicates that LLMs can support architectural recovery by adding semantic interpretation to the structural clusters generated by tools like ACDC, WCA, Limbo, and ARC. However, the quality of the output depends strongly on the prompt. A prompt that includes class names, dependencies, and cluster context is more likely to produce useful architectural descriptions than a generic prompt.

The next step is to compare this lightweight Colab-based model with the larger **zai-org/GLM-5** model on the HPC environment. This will help determine whether a larger model provides deeper architectural explanations, or whether careful prompt design is the main factor in producing useful results.

4.2 RQ2: How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

4.2.1 Model Choice and Practical Research Constraints

While RQ1 planned to use **zai-org/GLM-5** on HPC, Week 4 ended up using **Nemotron-70B** due to its availability on the cluster. This is a realistic challenge in academic research you don't always get to use the model you planned for. That said, Nemotron-70B proved capable enough for the task, and comparing its outputs against GLM-5 remains an interesting direction for future work.

4.2.2 Why the Switch from sbatch to srun Mattered

Moving from `sbatch` to `srun` with the `devel` queue turned out to be one of the most impactful decisions of the week. When debugging scripts and refining the workflow, waiting a long time for each batch job to start was slowing everything down.Interactive sessions let us catch errors quickly and move forward.This suggests that for any iterative LLM-based work on HPC, starting with a development queue is the smarter approach rather than going straight to large batch submissions.

4.2.3 What the Hierarchical Approach Tells Us

The bottom-up structure of the workflow processing individual files first, then subdirectories, then the full module turned out to be a natural fit for understanding a complex codebase. Trying to summarize 101 Java files in one go would have produced shallow output. Breaking it down level by level gave the model the right amount of context at each stage, which is closer to how a developer actually reads and understands an unfamiliar codebase.

4.2.4 Structural Clustering and LLM Summarization Work Better Together

Looking at the full picture across all weeks, structural clustering and LLM summarization clearly complement each other.Clustering tools like ACDC, WCA, and ARC group classes into components but cannot explain what those components actually do. The hierarchical LLM summarization fills that gap by producing readable descriptions of each subsystem. Combining both using cluster boundaries as additional context for the LLM would be a logical and promising next step.

5 Threats to Validity

While our results provide a solid baseline, we have to acknowledge the "gremlins in the machine" that might affect our findings. We've broken these down into three categories:

5.1 Construct Validity

Construct validity asks if our tools a2a and Cvg actually measured the "architectural similarity" we think they did. A major concern here is the zero Cvg scores. While the structural similarity (a2a) was high, the lack of overlap suggests that the metrics might be sensitive to how clusters are named or how entities are nested within the RSF files.

5.2 External Validity

This threat concerns whether our findings can be applied beyond our specific bubble. We focused exclusively on the Hadoop YARN ResourceManager Capacity Scheduler. Because Hadoop is a mature, highly decoupled system, the clustering behavior we observed might not hold true for smaller and monolithic projects. Our results are a snapshot of one component, not a universal rule for all software.

5.3 Reliability

For the ARCADE tools, the answer is likely yes, as the commands are deterministic. However, our LLM prototyping introduces a bit of randomness. Large Language Models like ByteDance-Seed/Seed-Coder-8BInstruct can be temperamental; a slight change in the prompt or even the temperature setting in Colab could lead to different interpretations of the Java code. While we verified the logic this week, the "human like" nature of the model means its reliability is only as good as the consistency of our prompts.

6 Conclusion and Future Work

Our analysis exposed a key challenge in software clustering: different algorithms can interpret the same codebase in fundamentally different ways. On paper, ACDC and WCA emerged as the most aligned pair, sharing the highest structural similarity score of 88.391. However, the data tells a double-sided story: while they agree on the general "shape" of the architecture, the low Coverage scores show that they still differ strongly in the exact class-level boundaries of their clusters. ACDC to WCA achieved a Coverage score of 0.1429, while WCA to ACDC achieved only 0.02, meaning the overlap is limited and asymmetric.

We also applied ARC clustering by combining semantic similarity from code embeddings with structural similarity from the RSF dependency data. The ARC result produced a different architectural view from the traditional ARCADE algorithms. Its A2A scores against ACDC, LIMBO, and WCA were much lower, around 26, showing that the semantic-structural clustering approach grouped the system differently from the purely structural clustering methods. The Coverage results were also mostly low, with only ARC vs LIMBO showing a small overlap, which suggests that ARC captures a different perspective on the Capacity Scheduler architecture.

Later, we extended the study by integrating hierarchical LLM-based architectural summarization using the **nvidia/Llama-3.1-Nemotron-70B-Instruct-HF** model on an HPC cluster. The generated architectural summary demonstrated that the LLM was able to recover meaningful subsystem responsibilities and inter-module relationships directly from the Hadoop YARN Capacity Scheduler source code. The model successfully identified major architectural areas such as allocation management, queue management, configuration handling, placement policies, and preemption mechanisms, while also describing how these sub-modules interact to support distributed resource scheduling. Compared to isolated file-level summaries, the hierarchical prompting strategy produced more coherent and system-level architectural explanations, suggesting that contextual prompting significantly improves the semantic quality of LLM-generated architectural recovery. These findings indicate that combining structural clustering techniques with hierarchical LLM summarization can provide both quantitative architectural views and human-readable semantic interpretations, creating a more complete software architecture recovery pipeline.

As future work, we plan to further evaluate the effectiveness of contextual hierarchical prompting against flat prompting using both qualitative analysis and automated semantic similarity metrics. We also aim to integrate the generated LLM summaries directly with ARC clustering outputs to create hybrid semantic-structural architectural diagrams. Another future direction involves testing additional open-source LLMs and larger embedding models to determine how model scale impacts architectural reasoning accuracy. Finally, expanding the study beyond the Capacity Scheduler to larger Hadoop YARN subsystems would help validate whether the observed clustering and summarization behaviors generalize to other large-scale distributed software systems.

7 Appendix

7.1 Teammates' Contribution

Teammate	Week 1	Week 2	Week 3	Week 4
Mihir Sane	4h	7h	10h	7h
Aryan Solanki	4h	7h	10h	7h
Sujit Maniya	4h	7h	10h	7h
Jenish Jai	4h	7h	10h	7h
Indudhara S	4h	7h	10h	7h

Table 3: Teammates' contribution summary

7.2 Shell Scripts

The complete workflow for both weeks was automated using two Bash shell scripts. The first script, `run_week1_arcade.sh`, sequentially executes all dependency extraction and clustering steps running the ARCADE JavaParser to generate the master RSF file, filtering it to the Capacity Scheduler package, and applying ACDC, WCA, and Limbo clustering producing all required output files in a single run. The second script, `run_week2_metrics_group10.sh`, automates the pairwise evaluation by running both the A2a and Cvg metric tools across all three algorithm pairs (ACDC vs Limbo, ACDC vs WCA, Limbo vs WCA) and saving the results to a consolidated output file for analysis. Both scripts include input validation and are fully reproducible given the same input JAR and ARCADE tool binaries. `run_week3_arc.sh` compares the Week 3 ARC clustering output against the Week 1 ACDC, WCA, and LIMBO outputs.

References

[1] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, *et al.*, "Apache hadoop yarn: Yet another resource negotiator," in *Proceedings of the 4th annual Symposium on Cloud Computing*, pp. 1–16, 2013.

[2] D. E. Perry and A. L. Wolf, "Foundations for the study of software architecture," *ACM SIGSOFT Software engineering notes*, vol. 17, no. 4, pp. 40–52, 1992.

[3] M. Schmitt Laser, N. Medvidovic, D. M. Le, and J. Garcia, "Arcade: an extensible workbench for architecture recovery, change, and decay evaluation," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1546–1550, 2020.

[4] V. Tzerpos and R. C. Holt, "Acdc: an algorithm for comprehension-driven clustering," in *Proceedings Seventh Working Conference on Reverse Engineering*, pp. 258–267, IEEE, 2000.

[5] P. Andritsos, P. Tsaparas, R. J. Miller, and K. C. Sevcik, "Limbo: Scalable clustering of categorical data," in *International Conference on Extending Database Technology*, pp. 123–146, Springer, 2004.

[6] T. Lutellier, D. Chollak, J. Garcia, L. Tan, D. Rayside, N. Medvidović, and R. Kroeger, "Measuring the impact of code dependencies on software architecture recovery techniques," *IEEE Transactions on Software Engineering*, vol. 44, no. 2, pp. 159–181, 2017.

[7] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pp. 31–53, IEEE, 2023.

[8] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, and J. Chen, "A survey of large language models for code: Evolution, benchmarking, and future trends," *arXiv preprint arXiv:2311.10372*, 2023.

[9] W. Hong, W. Yu, X. Gu, G. Wang, G. Gan, H. Tang, J. Cheng, J. Qi, J. Ji, L. Pan, *et al.*, "Glm-4.5 v and glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning," *arXiv preprint arXiv:2507.01006*, 2025.